Informe de Laboratorio: Dockerización por Perfiles

Estudiante:
Maia Saavedra
Fecha:
27 de octubre de 2026
Repositorio GitHub:
`/mxixsxxvedrx/seguimiento2Docker`

1. Arquitectura del Dockerfile (Ejercicio 5)

Para la resolución del Ejercicio 5, se diseñó un `Dockerfile` dinámico capaz de adaptarse a distintos perfiles de la aplicación. El objetivo principal es garantizar el principio de aislamiento: cada imagen debe contener exclusivamente las dependencias necesarias para su perfil, optimizando el almacenamiento.

```
FROM python:3.12-alpine
ARG PERFIL
ENV PERFIL=${PERFIL}
WORKDIR /app

# Copiar directorios y validación Fail-fast
COPY perfiles/ /app/perfiles/
RUN if [ -z "$PERFIL" ] || [ ! -d "/app/perfiles/$PERFIL" ]; then \
        echo "Error: Perfil no valido o no especificado." >&2; exit 1; fi

# Instalación aislada de dependencias
RUN pip install --no-cache-dir -r /app/perfiles/${PERFIL}/requirements.txt

COPY app/ /app/
CMD ["python", "main.py"]
```

2. Comandos de Construcción

Se generaron tres imágenes independientes inyectando el argumento de construcción (`--build-arg`) desde la terminal:

```
sudo docker build --no-cache --build-arg PERFIL=agenda -t evento-agenda .
sudo docker build --no-cache --build-arg PERFIL=banner -t evento-banner .
sudo docker build --no-cache --build-arg PERFIL=inscripcion -t evento-inscripcion .
```

3. Análisis Técnico e Implementación (Parte I)

Pregunta 1: ¿Cómo sabe main.py qué perfil ejecutar?

El script `main.py` está diseñado para leer las variables de entorno del sistema operativo utilizando `os.getenv("PERFIL")` en Python. Para que el contenedor posea esta variable, el `Dockerfile` captura el valor en tiempo de construcción mediante la instrucción `ARG PERFIL` y posteriormente lo persiste para el tiempo de ejecución (run-time) utilizando `ENV PERFIL=${PERFIL}`.

4. Análisis Técnico e Implementación (Parte II)

Pregunta 3: Comparación de tamaños de las imágenes generadas

Al ejecutar el comando de validación de imágenes, se obtuvieron los siguientes pesos:

- **evento-agenda:** 87.6 MB
- **evento-banner:** 96.1 MB
- **evento-inscripcion:** 88.9 MB

Análisis de eficiencia: La diferencia de megabytes entre las tres imágenes corrobora el éxito del aislamiento. Docker empaquetó de forma estricta únicamente las librerías declaradas en el `requirements.txt` de cada perfil, siendo la imagen del *banner* la más pesada debido a dependencias como `pyfiglet`.

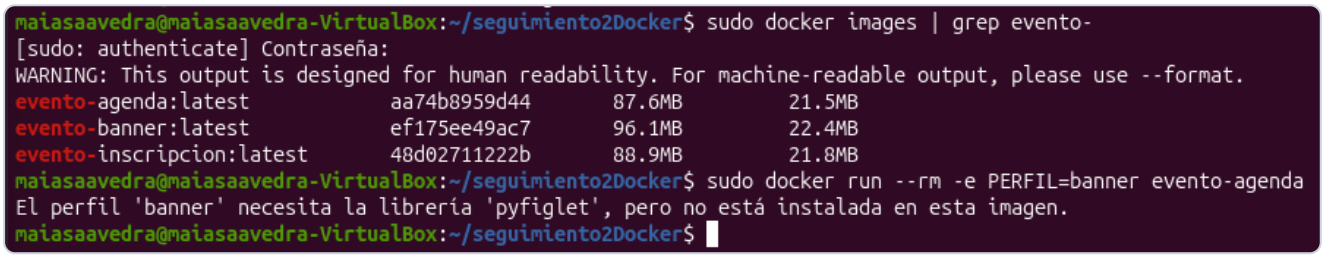
Pregunta 4: Prueba de seguridad de dependencias cruzadas

Al forzar la ejecución mediante el comando `sudo docker run --rm -e PERFIL=banner evento-agenda`, la aplicación se interrumpe y arroja una advertencia: *"El perfil 'banner' necesita la librería 'pyfiglet', pero no está instalada en esta imagen."*

> **Causa Raíz (Technical Insight):** Este fallo intencional demuestra la solidez de la arquitectura. La imagen `evento-agenda` se compiló exclusivamente con los requerimientos de la agenda. Al inyectarle `PERFIL=banner` en tiempo de ejecución, el sistema intenta ejecutar la lógica del banner, generando un error de importación debido a la ausencia de la librería correspondiente.

5. Evidencia Fotográfica de Ejecución

Captura de terminal comprobando los pesos de las imágenes (Pregunta 3) y el fallo provocado por dependencias cruzadas aisladas (Pregunta 4):



6. Control de Versiones (Git)

Todo el código fuente modificado, incluyendo los Dockerfiles de los 5 ejercicios, fue consolidado, subido (Push) y convertido a Pull Request de manera exitosa a la plataforma GitHub, en la rama designada `maia-saavedra`.